МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ

ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

КАФЕДРА ТЕОРЕТИЧЕСКОЙ И ПРИКЛАДНОЙ ИНФОРМАТИКИ

ЛАБОРАТОРНАЯ РАБОТА

по дисциплине «Введение в искусственный интеллект и логическое программирование»

ВСТРОЕННЫЕ ПРЕДИКАТЫ ПРОЛОГА. ПРЕДИКАТЫ ВВОДА-ВЫВОДА, ПРЕДИКАТЫ ОБРАБОТКИ СТРОК

НОВОСИБИРСК 2026

1. Цель работы

Приобретение навыков работы с внелогическими предикатами Пролога, изучение стандартных предикатов работы со строками и освоение методов обработки текстовых файлов с построчным чтением и записью данных.

2. Задание

Дан текстовый файл, разделенный на строки. Выравнивание строки заключается в том, что между ее отдельными словами дополнительно вносятся пробелы так, чтобы длина строки стала равной заданной длине (требуемая длина не меньше исходной), а последнее слово строки сдвинулось к ее правому краю. Написать программу выравнивания строк данного текста.

3. Описание разработанной программы

1) Структура базы знаний:
    - file = input_file; output_file — символьные имена файлов для открытия и работы с ними
    - wordlist = string* — рекурсивная структура списка строк (слов), используется для представления строки в виде набора слов
2) Основные предикаты:
    - process — главная процедура, управляющая процессом: запрос длины строки, открытие файлов, запуск обработки, закрытие файлов
    - read_and_process(integer) — рекурсивное чтение строк из входного файла и запись выровненных строк в выходной файл до достижения пустой строки
    - justify_string(string, integer, string) — выравнивание одной строки до заданной длины
    - split_string(string, wordlist) — разбиение строки на список отдельных слов
    - count_chars(wordlist, integer) — подсчёт суммарной длины всех слов в списке (без пробелов)
    - distribute_spaces(wordlist, integer, wordlist) — равномерное распределение заданного количества пробелов между словами в списке
    - distribute_evenly(wordlist, integer, integer, integer, wordlist) — распределение пробелов с учётом базового количества и дополнительных пробелов
    - make_space_block(integer, string) — создание блока (строки) из указанного количества пробелов

- length_of_list(wordlist, integer) — подсчёт количества элементов в списке
- assemble_string(wordlist, string) — сборка списка строк (слов и пробелов) в единую результирующую строку
- add_spaces_to_end(string, integer, string) — добавление заданного количества пробелов в конец строки

3) Принцип работы:

Программа запрашивает требуемую длину строки, открывает input.txt для чтения и output.txt для записи. С помощью рекурсивного предиката read_and_process строки читаются и обрабатываются до пустой строки. Для каждой строки вызывается justify_string, реализующий алгоритм выравнивания.

Случай 1 — несколько слов. Строка разбивается на слова, вычисляется суммарная длина слов и требуемое число пробелов. Пробелы равномерно распределяются между словами: вычисляется базовое количество и остаток, распределяемый по одному в первые промежутки. Блоки пробелов создаются рекурсивно, чередуются со словами, результат собирается в строку.

Случай 2 — одно слово или пустая строка. Пробелы добавляются в конец строки.

4. Примеры запросов к базе знаний и ее ответов

Исходный файл:

```
Goodbye, America, o-o...
Gde ya ne budu nicogda
Uslyshu li pesnyu
Kotoruyu zapomnyu navsegda
```

Длина строки – 30:

```
Goodbye , America , o - o . ..
Gde   ya   ne   budu   nicogda
Uslyshu       li       pesnyu
Kotoruyu   zapomnyu   navsegda
```

Длина строки – 50:

```
Goodbye    ,   America   ,   o   -   o   .   .   .
Gde        ya        ne        budu        nicogda
Uslyshu                li                  pesnyu
Kotoruyu            zapomnyu            navsegda
```

5. Листинг разработанной базы знаний на языке Пролог

```
DOMAINS
    file = input_file; output_file
    wordlist = string*

PREDICATES
    nondeterm process
    justify_string(string, integer, string)
    split_string(string, wordlist)
    count_chars(wordlist, integer)
    distribute_spaces(wordlist, integer, wordlist)
    assemble_string(wordlist, string)
    add_spaces_to_end(string, integer, string)
    make_space_block(integer, string)
    length_of_list(wordlist, integer)
    distribute_evenly(wordlist, integer, integer, integer, wordlist)
    nondeterm read_and_process(integer)

CLAUSES

    process :-
        write("Введите требуемую длину строки: "),
        readint(MaxLen),
        openread(input_file, "input.txt"),
        readdevice(input_file),
        openwrite(output_file, "output.txt"),
        writedevice(output_file),
        read_and_process(MaxLen),
        writedevice(screen),
        closefile(input_file),
        closefile(output_file),
        write("Выровненные строки в output.txt"), nl.

    read_and_process(MaxLen) :-
        readln(Line),
        justify_string(Line, MaxLen, ResLine),
        write(ResLine), nl,
        not(Line = ""),
        read_and_process(MaxLen).

    read_and_process(_).

    % Neskolko slov
    justify_string(Line, MaxLen, ResLine) :-
        split_string(Line, Words),
        Words = [_, _ | _], !,
        count_chars(Words, TotalChars),
        TotalSpaces = MaxLen - TotalChars,
        distribute_spaces(Words, TotalSpaces, SpacedWords),
        assemble_string(SpacedWords, ResLine).

    % Odno slovo ili pustaya stroka
    justify_string(Line, MaxLen, ResLine) :-
        str_len(Line, Len),
        Spaces = MaxLen - Len,
        add_spaces_to_end(Line, Spaces, ResLine).

    % Dobavlenie probelov v konec stroki
    add_spaces_to_end(Line, 0, Line) :- !.
    add_spaces_to_end(Line, N, ResLine) :-
        N > 0,
        concat(Line, " ", Tmp),
        N1 = N - 1,
```

```
        add_spaces_to_end(Tmp, N1, ResLine).

    % Razbienie stroki na spisok slov
    split_string("", []) :- !.
    split_string(Str, [Token | RestTokens]) :-
        fronttoken(Str, Token, Rest),
        Token <> "", !,
        split_string(Rest, RestTokens).
    split_string(_, []) :- !.

    % Podschet summy dlin vseh slov v spiske
    count_chars([], 0).
    count_chars([H | T], Total) :-
        str_len(H, Len),
        count_chars(T, Rest),
        Total = Len + Rest.

    % Ravnomernoe raspredelenie probelov mezhdu slovami
    distribute_spaces(Words, TotalSpaces, SpacedWords) :-
        length_of_list(Words, WordCount),
        GapsCount = WordCount - 1,
        BaseSpaces = TotalSpaces div GapsCount,
        ExtraSpaces = TotalSpaces mod GapsCount,
        distribute_evenly(Words, GapsCount, BaseSpaces, ExtraSpaces, SpacedWords).

    % Podschet dliny spiska
    length_of_list([], 0).
    length_of_list([_ | T], N) :-
        length_of_list(T, N1),
        N = N1 + 1.

    % Sozdanie bloka iz N probelov
    make_space_block(0, "") :- !.
    make_space_block(N, Block) :-
        N > 0,
        N1 = N - 1,
        make_space_block(N1, Tmp),
        concat(" ", Tmp, Block).

    % Ostalos odno slovo - vyhod
    distribute_evenly([W], _, _, _, [W]) :- !.

    % Est dopolnitelnye probely
    distribute_evenly([W1, W2 | Rest], GapsLeft, Base, Extra, [W1, Block | Padded]) :-
        Extra > 0, !,
        SpaceCount = Base + 1,
        make_space_block(SpaceCount, Block),
        Extra1 = Extra - 1,
        distribute_evenly([W2 | Rest], GapsLeft, Base, Extra1, Padded).

    % Dop probely zakonchilis, dobavlyaem bazovoe kolichestvo
    distribute_evenly([W1, W2 | Rest], GapsLeft, Base, 0, [W1, Block | Padded]) :-
        make_space_block(Base, Block),
        distribute_evenly([W2 | Rest], GapsLeft, Base, 0, Padded).

    % Sborka spiska strok v odny stroky
    assemble_string([], "").
    assemble_string([H | T], Res) :-
        assemble_string(T, Tmp),
        concat(H, Tmp, Res).

GOAL
    process.
```